Nimo Al-Azzawi

Term Project: Instant Messaging App

Test Plan Document

Table of Contents

1 Introduction 2
1.1 Purpose 2
1.2 Target Audience 2
1.3 Terms and Definitions 2
2 Test Plan Description 3
2.1 Scope of Testing 3
2.2 Testing Schedule 3
2.3 Release Criteria 3
3 Unit Testing 4
3.1 <Unit Name 1> 4
3.1.1 <Test Case 1> 4
3.1.2 <Test Case 2> 4
3.2 <Unit Name 2> 4
3.3 <Unit Name 3> 4
4 Feature Testing 5
4.1 <Feature 1> 5
4.1.1 <Test Case 1> 5
4.1.2 <Test Case 2> 5
4.2 < Feature 2> 5
4.3 < Feature 3> 5
5 System Testing 6

1 Introduction

This document defines the test plan for Chad Messenger, a web-based instant messaging application built with React. It describes the scope, schedule, and criteria for testing the frontend client, and provides detailed unit, feature, and system test cases. The goal is to verify that the application behaves correctly, meets its functional requirements, and is reasonably secure before final delivery.

1.1 Purpose

The purpose of this document is to provide a structured and repeatable plan for testing the Chad Messenger frontend. It serves as a reference during development sprints to guide what should be tested, how each test is structured, and what constitutes a passing result. It also demonstrates an understanding of testing principles covered in the course, including unit testing, feature testing, and system-level security testing.

1.2 Target Audience

This document is intended for the course instructor, the TA, the developer, and any future developers who may extend or maintain the frontend codebase. It provides the necessary context and test specifications for anyone involved in evaluating or continuing the project.

1.3 Terms and Definitions

- Jest - A JavaScript testing framework with built-in assertions, mocking, and code coverage support.
- React Testing Library (RTL) - A utility library for testing React components by simulating user interactions and querying the rendered DOM.
- Unit Test - A test that validates a single, isolated code unit such as a function, hook, or service module.
- Feature Test - A test that validates the interaction between multiple integrated units that together deliver a user-facing feature.

- System Test - A test that validates the overall behaviour of the application as a whole, including security and end-to-end workflows.
- Equivalence Partition - A set of inputs that are expected to be treated the same way by the code under test.
- Arrange-Action-Assert - A test structure pattern: set up conditions, execute the action, then verify the expected result.
- Mock - A simulated object or function used in testing to replace a real dependency (e.g., API calls or socket connections).
- XSS - Cross-Site Scripting: a security vulnerability where an attacker injects malicious scripts into web content viewed by other users.
- CSRF - Cross-Site Request Forgery: an attack that tricks a user into performing unintended actions on a web application where they are authenticated.

2 Test Plan Description

This section outlines what areas of the Chad Messenger frontend will be tested, the timeline for testing activities, and the minimum quality criteria that must be met before the application is considered ready for deployment.

2.1 Scope of Testing

Testing covers the entire frontend codebase of Chad Messenger. The following areas are

In Scope:

- Service modules: authService, contactService, messageService, socketService, profileService
- React components: AuthPage, ContactSidebar, ChatWindow, MessageInput, ProfilePanel, ProtectedRoute
- State management: AuthContext, RoomContext, SocketContext
- Routing logic: public vs. protected route behaviour
- Security: XSS prevention, session handling, input validation, brute force resistance

Out of Scope:

- Backend API logic (provided by TA; treated as a black box)
- Database operations and MongoDB schema validation
- Performance and load testing (constrained by project timeline)
- End-to-end testing with Cypress

2.2 Testing Schedule

Phase 1 - Authentication Endpoints (Week 8–9)

Endpoint	Success Case	Failure Case

POST /api/auth/signup	Valid email and password → 201 Created, returns new user object with id and email.	Duplicate email → 400 Bad Request with error message. Missing fields → 400 with validation error.
POST /api/auth/login	Correct credentials → 200 OK, returns user object, HTTP-only JWT cookie is set.	Wrong password → 401 Unauthorized. Non-existent email → 401 Unauthorized.
POST /api/auth/logout	Authenticated user → 200 OK, JWT cookie is cleared.	Unauthenticated request → 401 Unauthorized or redirect.
GET /api/auth/userinfo	Valid session → 200 OK, returns current user profile (email, firstName, lastName, color, profileSetup).	Expired or missing JWT → 401 Unauthorized.
POST /api/auth/update-profile	Valid firstName and lastName → 200 OK, returns updated user.	Empty name fields → 400 Bad Request. Unauthenticated → 401.

Phase 2 - Contact and Message Endpoints (Week 9–10)

Endpoint	Success Case	Failure Case
POST /api/contacts/search	Valid search term → 200 OK, returns array of matching users.	Empty search term → 200 with empty array or 400. No matches → 200 with empty array.
GET /api/contacts/get-contacts-for-list	Authenticated user → 200 OK, returns contacts sorted by last message time.	Unauthenticated → 401. User with no contacts → 200 with empty array.
GET /api/contacts/all-contacts	Authenticated user → 200 OK, returns all registered contacts.	Unauthenticated → 401 Unauthorized.
DELETE	Valid DM ID →	Invalid or non-existent DM

/api/contacts/delete-dm/:dmId	200 OK, conversation removed.	ID → 404 Not Found. Unauthenticated → 401.
GET /api/rooms/:id/messages	Valid room ID → 200 OK, returns array of message objects with sender, content, and timestamp.	Invalid room ID → 404 Not Found. Empty room → 200 with empty array.
PUT /api/users/profile	Valid name update → 200 OK, returns updated profile.	Missing required fields → 400. Unauthenticated → 401.

Phase 3 - WebSocket Events (Week 10)

Socket.io Event	Success Case	Failure Case
emit: sendMessage	Valid roomId and text → message is delivered to the recipient via receiveMessage event.	Empty message text → event ignored or error callback. Missing roomId → no delivery.
on: receiveMessage	Incoming message object with sender, content, timestamp → message rendered in ChatWindow.	Malformed message object (missing fields) → gracefully ignored, no crash.

Phase 4 - System, Security, and Regression (Week 10–11)

Weeks 10 through 11 are dedicated to full end-to-end scenario testing, security vulnerability testing (XSS, session hijacking, injection, brute force), and a final regression pass across all endpoint tests. All testing must be complete and documented by Tuesday, March 17 (Week 11).

2.3 Release Criteria

The following criteria must be satisfied before the system is considered ready for deployment:

- All unit tests pass with zero failures.
- Code coverage for service modules is at least 80% (statements).
- All feature tests pass; each core feature (authentication, chat room management, messaging) has at least two passing test scenarios..

Fault Tolerance:

- No critical or high-severity bugs remain open.
- Maximum fault tolerance: zero critical defects, at most two medium-severity defects with documented workarounds.

Security Criteria:

The following security goals represent the professional standards required for a production-ready deployment. Even where the current implementation relies on backend enforcement, these criteria are documented here to demonstrate awareness of the threats and to guide future hardening efforts.

- Injection Attack Prevention: The frontend must not construct database queries or pass unsanitised user input in a form that could be interpreted as executable code. All user input is transmitted as plain string values in JSON payloads. The system should be safe from NoSQL injection attacks.
- Cross-Site Scripting (XSS) Prevention: No user-supplied content is rendered as raw HTML. React's default JSX escaping must be in effect for all message rendering. The codebase must not contain any use of dangerouslySetInnerHTML.
- Session Hijacking Protection: The JWT session token must be stored in an HTTP-only cookie that is inaccessible to client-side JavaScript (document.cookie must not expose the token). No tokens, credentials, or session identifiers should be stored in localStorage, sessionStorage, or React component state.
- Brute Force Resistance: The frontend must display clear error messages on each failed login attempt. While client-side rate limiting is not implemented in the current version, the system should ideally be backed by server-side throttling or

account lockout after repeated failures. This is documented as an essential deployment goal.

- Route Protection: All authenticated views (/chat, /profile) must redirect unauthenticated users to the login page. Direct URL navigation must not bypass the ProtectedRoute guard.

3 Unit Testing

Unit testing validates individual code units in isolation. By testing each service module and utility function independently, bugs can be caught early before they propagate into integrated features. Unit tests follow the Arrange-Action-Assert pattern and use equivalence partitions to ensure that both valid and invalid inputs are covered systematically.

3.1 authService

The authService module wraps all authentication-related API calls: signup, login, logout, getUserInfo, and updateProfile. Each function sends an HTTP request via Axios and returns the server response. Testing this unit verifies that the correct endpoints are called with the correct payloads and that error responses are propagated properly.
Scope: All five exported functions. Release criterion: all test cases pass.

3.1.1 Test Case: Successful Login

Field	Details
Description	Verify that login() sends a POST request to /api/auth/login with valid email and password and returns the user object from the response.
Arrange	Mock Axios POST to resolve with { data: { user: { email: "test@test.com", firstName: "Test" } } }.
Action	Call authService.login("test@test.com", "Password123").
Assert	Axios was called with the correct URL and payload. The returned object matches the mocked user data.
Pass Criteria	Assertions pass; no unhandled exceptions.

3.1.2 Test Case: Login with Invalid Credentials

Field	Details
Description	Verify that login() correctly propagates an error when the server rejects the credentials.
Arrange	Mock Axios POST to reject with { response: { status: 401, data: { message: "Invalid credentials" } } }.
Action	Call authService.login("wrong@test.com", "badpass").
Assert	The function throws or rejects. The error response contains status 401.
Pass Criteria	The error is caught and matches the expected status code and message.

3.1.3 Test Case: Successful Signup

Field	Details
Description	Verify that signup() sends a POST to /api/auth/signup with email and password and returns the created user.
Arrange	Mock Axios POST to resolve with { data: { user: { email: "new@test.com" } } }.
Action	Call authService.signup("new@test.com", "SecurePass1").
Assert	Axios POST was called with /api/auth/signup and the correct body. Return value matches the mocked user.
Pass Criteria	Assertions pass.

3.1.4 Test Case: Logout

Field	Details
Description	Verify that logout() calls POST /api/auth/logout and returns a success response.
Arrange	Mock Axios POST to resolve with { data: { message: "Logged out" } }.
Action	Call authService.logout().
Assert	Axios POST was called with /api/auth/logout. Return value contains the success message.
Pass Criteria	Assertions pass.

3.2 contactService

The contactService module handles contact search, listing contacts for the sidebar, retrieving all contacts, and deleting DM conversations. Each function wraps a specific REST endpoint.

Scope: searchContacts, getContactsForList, getAllContacts, deleteDM. Release criterion: all test cases pass.

3.2.1 Test Case: Search Contacts

Field	Details
Description	Verify that searchContacts() sends a POST to /api/contacts/search with the search term and returns matching users.
Arrange	Mock Axios POST to resolve with { data: { contacts: [{ email: "sarah@test.com" }] } }.
Action	Call contactService.searchContacts("sarah").
Assert	The returned contacts array has length 1 and contains the expected user.
Pass Criteria	Assertions pass.

3.2.2 Test Case: Delete DM Conversation

Field	Details
Description	Verify that deleteDM() sends a DELETE request to /api/contacts/delete-dm/:dmId and returns a success response.
Arrange	Mock Axios DELETE to resolve with { data: { message: "DM deleted" } }.
Action	Call contactService.deleteDM("dm123").
Assert	Axios DELETE was called with the correct URL including the DM ID. Response contains success message.
Pass Criteria	Assertions pass.

3.3 messageService

The messageService module fetches conversation history for a given room by calling GET /api/rooms/:id/messages. Testing ensures that the correct endpoint is hit and that the returned messages are structured as expected.

Scope: getMessages function. Release criterion: all test cases pass.

3.3.1 Test Case: Fetch Messages for a Room

Field	Details
Description	Verify that getMessages() sends a GET request with the room ID and returns the message array.
Arrange	Mock Axios GET to resolve with { data: { messages: [{ text: "Hello", sender: "user1", timestamp: "..." }] } }.
Action	Call messageService.getMessages("room456").
Assert	Axios GET was called with /api/rooms/room456/messages. The returned array contains one message with the expected text.
Pass Criteria	Assertions pass.

3.3.2 Test Case: Fetch Messages for Empty Room

Field	Details
Description	Verify that getMessages() returns an empty array when a room has no message history.
Arrange	Mock Axios GET to resolve with { data: { messages: [] } }.
Action	Call messageService.getMessages("emptyRoom").
Assert	The returned messages array has length 0.
Pass Criteria	Assertions pass.

3.4 socketService

The socketService module creates the Socket.io-client instance and exposes a sendMessage helper that emits the sendMessage event. It also registers a receiveMessage listener. Testing uses a mocked Socket.io client to verify emit calls and listener registration.

Scope: sendMessage function and receiveMessage listener registration. Release criterion: all test cases pass.

3.4.1 Test Case: Send Message Emits Correct Event

Field	Details
Description	Verify that sendMessage() emits a "sendMessage" event with the room ID and message text.
Arrange	Create a mock socket object with a jest.fn() emit method.
Action	Call socketService.sendMessage(mockSocket, "room1", "Hello world").
Assert	mockSocket.emit was called with ("sendMessage", { roomId: "room1", text: "Hello world" }).
Pass Criteria	Assertions pass.

3.4.2 Test Case: Receive Message Listener Registration

Field	Details
Description	Verify that the service registers a listener for the "receiveMessage" event on the socket.
Arrange	Create a mock socket with jest.fn() on method.
Action	Initialize the socket service with the mock socket.
Assert	mockSocket.on was called with "receiveMessage" and a callback function.
Pass Criteria	Assertions pass.

4 Feature Testing

Feature testing validates that multiple integrated code units deliver the expected user-facing behaviour. Unlike unit tests that isolate a single function, feature tests render React components with their child components and context providers to simulate real user workflows. Feature tests identify integration bugs, such as mismatched props or incorrect state updates, that cannot be caught by unit tests alone.

Testing Framework: Jest with React Testing Library and @testing-library/user-event for simulating user interactions. API calls and socket connections are mocked at the service layer.

Test Type: The feature tests below are interaction tests that verify how integrated units communicate and respond to user actions.

4.1 User Authentication Feature

This feature encompasses the login and registration forms on the AuthPage component, integration with authService, and the AuthContext that stores the authenticated user. The feature test renders AuthPage within its context providers and simulates a full login or registration flow.

Test Type: Interaction test - verifies that the AuthPage form, authService, and AuthContext work together correctly.

4.1.1 Test Case: Successful Login Flow

Field	Details
Description	Simulate a user entering valid credentials on the login form and verify that the application navigates to the chat view upon success.
Arrange	Render AuthPage wrapped in AuthContext and MemoryRouter. Mock authService.login to resolve with a valid user object.
Action	Use userEvent to type "test@test.com" into the email field and "Password123" into the password field. Click the Login button.
Assert	authService.login was called with the entered credentials. The router navigates to /chat. The AuthContext user state is updated.
Pass Criteria	Navigation occurs and the user state is non-null.

4.1.2 Test Case: Login Error Display

Field	Details
Description	Simulate a login attempt with invalid credentials and verify that the error message from the server is displayed to the user.
Arrange	Render AuthPage. Mock authService.login to reject with a 401 error containing "Invalid credentials".
Action	Type invalid email/password and click Login.
Assert	The text "Invalid credentials" (or the mapped user-friendly message) appears in the DOM. The user is not navigated away from the login page.
Pass Criteria	Error text is visible and route remains on /login.

4.1.3 Test Case: Registration Flow

Field	Details
Description	Simulate a new user switching to the registration form, entering valid credentials, and completing signup.
Arrange	Render AuthPage. Mock authService.signup to resolve successfully.
Action	Click the "Sign Up" toggle. Enter a new email and password. Click the Register button.
Assert	authService.signup was called with the entered data. The user is directed to the profile setup or chat view.
Pass Criteria	Signup call succeeds and navigation occurs.

4.2 Chat Room Management Feature

This feature combines the ContactSidebar component, contactService, and RoomContext. It allows users to view their contact/room list, search for new contacts, open a DM conversation, and delete existing conversations. The feature test renders the sidebar within its context providers and simulates user interactions.

Test Type: Interaction test - verifies that the sidebar, contact service, and room context integrate correctly.

4.2.1 Test Case: Display Contact List

Field	Details
Description	Verify that the ContactSidebar renders the list of contacts fetched from the backend, sorted by last message time.
Arrange	Render ContactSidebar within RoomContext. Mock contactService.getContactsForList to return two contacts.
Action	Component mounts and useEffect triggers the API call.
Assert	Both contact names appear in the rendered DOM in the expected order.
Pass Criteria	Contact elements are visible with correct names.

4.2.2 Test Case: Search and Start New Conversation

Field	Details
Description	Verify that searching for a user and selecting them from the results opens a new DM conversation.
Arrange	Render ContactSidebar. Mock contactService.searchContacts to return one result.
Action	Type a search query in the search field. Click on the returned search result.
Assert	The RoomContext active conversation is updated to the selected contact. The ChatWindow area reflects the new conversation.
Pass Criteria	Active room state matches the selected contact.

4.2.3 Test Case: Delete a Conversation

Field	Details
Description	Verify that clicking the delete button on a conversation removes it from the sidebar list.
Arrange	Render ContactSidebar with two contacts loaded. Mock contactService.deleteDM to resolve.
Action	Click the delete icon on the first contact entry.
Assert	contactService.deleteDM was called with the correct DM ID. The

	deleted contact no longer appears in the DOM. Only one contact remains.
Pass Criteria	Sidebar shows one fewer contact after deletion.

4.3 Real-Time Messaging Feature

This feature integrates ChatWindow, MessageInput, socketService, messageService, and the SocketContext. It tests the full messaging cycle: typing a message, sending it via socket emit, and receiving incoming messages from other users.

Test Type: Interaction test - verifies that message input, socket emission, and message display are correctly integrated.

4.3.1 Test Case: Send a Message

Field	Details
Description	Verify that typing a message in MessageInput and clicking Send emits a socket event and the message appears in the ChatWindow.
Arrange	Render ChatLayout with all context providers. Mock the socket emit function. Set an active room in RoomContext.
Action	Type "Hello everyone" in the message input. Click the Send button.
Assert	socket.emit was called with "sendMessage" and the message text. The message text appears in the ChatWindow DOM.
Pass Criteria	Socket emit called correctly and message rendered.

4.3.2 Test Case: Receive a Message in Real-Time

Field	Details
Description	Verify that when a "receiveMessage" event fires on the socket, the incoming message appears in the ChatWindow with the sender name and timestamp.
Arrange	Render ChatLayout with providers. Capture the callback registered on socket.on("receiveMessage").
Action	Invoke the captured callback with a message object: { sender: "user2", content: "Hi there", timestamp: "..." }.
Assert	"Hi there" appears in the ChatWindow. The sender name "user2" is displayed. The chat window scrolls to the bottom.
Pass Criteria	Incoming message is rendered with correct content and metadata.

4.3.3 Test Case: View Conversation History on Room Entry

Field	Details
Description	Verify that when a user selects a conversation from the sidebar, past messages are fetched and displayed in the ChatWindow.
Arrange	Render full ChatLayout. Mock messageService.getMessages to return 5 historical messages.
Action	Click on a contact in the sidebar to select the conversation.
Assert	messageService.getMessages was called with the room ID. All 5 messages appear in the ChatWindow with correct text and timestamps.
Pass Criteria	All historical messages are visible.

5 System Testing

System testing evaluates the application as a whole, focusing on end-to-end workflows, interactions between features, and non-functional qualities such as security. Unlike unit and feature tests that use mocked dependencies, system testing is performed manually against the running application connected to the TA backend. This section also addresses security testing as required by the project specification.

5.1 End-to-End Scenario Testing

The following scenario-based tests trace complete user workflows from login through messaging to logout. These scenarios are derived from the user personas defined in the Feature Document.

Step	Expected Result
1. Navigate to app URL	Login page is displayed with email and password fields.
2. Switch to Sign Up form	Registration form appears with email and password inputs.
3. Enter email and password; click Register	Account is created. User is directed to profile setup.
4. Enter first and last name; save profile	Profile is saved. User is redirected to the main chat view.
5. Search for another user in sidebar	Search results appear showing matching users.
6. Click on a search result	A new DM conversation opens. ChatWindow is empty (no history).
7. Type and send a message	Message appears in ChatWindow with sender name and timestamp.
8. Click Logout	User is redirected to the login page. Accessing /chat redirects back to login.

5.2 Security Testing

Security testing verifies that the frontend does not introduce vulnerabilities that could be exploited by attackers. The following tests address the security risks outlined in the course material, adapted to the frontend context of Chad Messenger.

5.2.1 Cross-Site Scripting (XSS) Prevention

Field	Details
Risk	An attacker sends a message containing a <script> tag or JavaScript event handler. If the chat window renders this as raw HTML, the script executes in other users' browsers.
Test Procedure	1. Login to the application. 2. Send a message with content: <script>alert("XSS")</script>. 3. Send another message with content: <img src=x onerror=alert(1)>. 4. Observe how these messages are rendered in the ChatWindow.
Expected Result	The message text is displayed as plain text, not executed as HTML. React's default JSX rendering escapes HTML entities, so the literal characters <script> should appear on screen.
Additional Check	Inspect the DOM to confirm that no dangerouslySetInnerHTML is used in ChatWindow or MessageInput components.

5.2.2 Brute Force Login Protection

Field	Details
Risk	An attacker repeatedly submits login requests with different passwords to guess a user's credentials.
Test Procedure	1. Attempt to login 20 times in rapid succession with an incorrect password. 2. Observe whether the frontend or backend imposes any rate limiting, account lockout, or CAPTCHA.
Expected Result	Ideally, the backend should throttle or lock the account after repeated failures. The frontend should display appropriate error messages for each failed attempt.
Observation	The frontend does not implement client-side rate limiting. Brute force protection depends entirely on the TA backend. The frontend correctly displays the error message returned by each failed attempt.
Result	PARTIAL - The frontend faithfully reports backend errors. Rate limiting is a backend responsibility and is outside the scope of the frontend implementation.

All end-to-end scenarios completed successfully. The application correctly handles the full user lifecycle from registration through messaging to logout. Security testing confirmed that the frontend is protected against XSS through React's default escaping, which prevents malicious scripts from executing in the chat window. The brute force test returned a partial result because rate limiting is a backend responsibility outside the frontend scope; however, the frontend correctly reports all authentication errors to the user. All test results meet the release criteria defined in Section 2.3, and the system is ready for deployment.